What I Learned from Building a Route Planner in CPT204

Project Context

Our project was a route planner for road trips in the United States. The user could choose a starting city, a destination, and optional attractions. The program then calculated a route and displayed the total distance.

At first, the task sounded like a normal programming assignment. However, after reading the task sheet carefully, we realized that the coursework was much broader. We had to design the program, implement algorithms, test the results, analyze sorting performance, write a formal report, prepare slides, record a video, and reflect on teamwork and project management.

This is why my sharing was not meant to be a technical defense. I used the technical project as an example to talk about coursework strategy, problem solving, communication, and preparation.

1. Read the Whole Task Before Coding

The first lesson is to read the whole task sheet before writing code.

It is easy to think, "This is a Java coursework, so I should start coding immediately." I had the same feeling at the beginning. But the task sheet showed that code was only one part of the submission. We also needed a Word report, a PowerPoint file, and a video presentation. The report had to explain program design, graph algorithms, sorting algorithm performance, testing results, teamwork, and contribution.

This changed the way we planned the project. If we only focused on the code, we might finish the program but still have no time or evidence for the report and presentation. Many marks came from explanation, analysis, and proof of work, not only from whether the program could run.

My advice is simple: before coding, identify all deliverables and marking criteria. Ask yourself what evidence you will need later, such as screenshots, test outputs, diagrams, or performance results. Understanding the task is already part of solving the problem.

2. Break a Large Requirement into Small Questions

The second lesson is to break a large task into smaller questions.

"Build a route planner" sounds like one big requirement, but in practice it contains many smaller problems. We needed to read CSV files, store city and road data, connect attractions to cities, choose a route algorithm, and display the result clearly.

The key step was modeling. We treated cities as nodes and roads as edges, so the route planner became a graph problem. After that, the task became easier to divide:

- How do we read the road and attraction data?
- How do we represent the graph?
- How do we map attractions to cities?
- How do we calculate the route?
- How do we show the result to the user?

This method reduced pressure. Instead of asking, "How do we build the whole system?", we could focus on one smaller problem at a time. Smaller problems are easier to start, easier to assign, easier to test, and easier to explain in the report.

3. Discuss Ideas, but Keep Academic Integrity

One important challenge was the algorithm choice.

At first, we mainly considered algorithms that were more familiar, such as Dijkstra's algorithm. Dijkstra is useful for finding the shortest path from one starting point. However, our project also allowed users to choose attractions, which meant the route might need to pass through several places before reaching the destination.

During the project, I discussed the problem with a friend from ICS. The discussion was not about sharing code or giving a complete solution. It was about possible ideas. From that conversation, I learned about Floyd-Warshall, an algorithm that can calculate shortest paths between all pairs of nodes. This gave us a new direction to explore.

The important point is the boundary. Friendly academic discussion is useful, especially when you are stuck. But there is a clear difference between discussing ideas and copying work. You can ask for general hints, learn concepts, and discuss possible directions. You should not copy other people's code or submit something you do not understand.

For me, this was an important lesson: academic integrity does not mean working in silence, but it does mean that the final implementation, understanding, and explanation must be your own.

4. Choose a Practical Solution, Not the Most Impressive One

Another challenge was visualization.

At the beginning, we considered using Google Maps API because it sounded more professional. A real map interface would look impressive. But after testing and discussion, we found that it was not very practical for this coursework. It could require API access, network connection, extra setup, and more time to learn. It also added uncertainty that we did not need.

Instead, we used JavaFX and coordinate mapping. Based on previous JavaFX experience, we mapped city latitude and longitude data onto a canvas and drew cities, roads, and routes ourselves.

This solution was not as fancy as a real map API, but it was clear, controllable, and suitable for the assignment. More importantly, we understood how it worked and could explain it in the report and presentation.

The lesson is that a good coursework solution is not always the most advanced-looking solution. It should be realistic, stable, explainable, and possible to finish before the deadline.

5. Teamwork Is Continuous Communication

Another lesson is about teamwork.

Before this coursework, it was easy to think teamwork meant dividing the task and combining everything at the end. In a programming project, that is risky. If two people work separately for too long, their parts may not fit together.

In our project, we used tools such as Trello and Git. Trello helped us track tasks, bugs, and testing progress. Git helped us manage versions and combine changes. However, the tools were not the most important part. The most important part was communication.

When one of us met a problem, we needed to say it early. When one part changed, we needed to tell the other person. When a task became too large, we needed to split it again.

My advice is: do not disappear when you are stuck. A small problem discussed early is much easier to solve than a large problem discovered one day before the deadline.

6. Save Evidence While You Work

The report and presentation should not be left until the end.

For this coursework, the report required screenshots, test results, algorithm explanations, design decisions, and reflections. The presentation also needed a clear story. If you only start preparing these materials after the code is finished, you may forget important details or struggle to reproduce results.

It is better to save evidence during development:

- Save screenshots when the interface works.
- Save test outputs when algorithms are tested.
- Write down important bugs and how they were fixed.
- Record why one design choice was made instead of another.
- Keep useful notes for the report and presentation.

Documentation is not something after the project. It is part of the project.

7. Three Suggestions for Junior Students

If I summarize my experience into three suggestions, they are:

Start early

Large coursework always has hidden work. Code may be only one part. Reports, slides, videos, testing, and reflection all take time.

Break it down

When a task looks too large, divide it into smaller questions. Small problems are easier to solve, easier to assign, and easier to explain.

Communicate actively

Talk to your teammate. Discuss problems early. Ask for general ideas when you are stuck, but respect academic integrity and make sure the final work is your own.

Final Takeaway

Looking back, the biggest gain from CPT204 Coursework 3 was not only learning an algorithm or improving Java coding ability. The bigger gain was learning how to plan a project, make practical decisions, work with others, and explain the process clearly.

Do not treat this coursework as only a programming task. Treat it as a chance to practice how real software work happens.

One-Minute Summary

This project taught me that a large coursework is not only about writing code. It is about understanding the task, breaking the problem down, making practical choices, communicating with teammates, and preparing evidence throughout the process. The most important lesson was learning to think more like a developer, not just a programmer.